O Efeito do Escalonamento do Meta-Alvo na Performance de Sistemas de Meta-Aprendizagem para Seleção de Algoritmos

Lael de Lima Santa Rosa[1]

[1]Instituto de Computação – Universidade Federal de Alagoas (UFAL)
Maceió – AL – Brasil

llsr@ic.ufal.br

Resumo. *A seleção de algoritmos em aprendizado de máquina frequentemente utiliza meta-aprendizagem, onde um meta-modelo aprende a prever o desempenho de algoritmos base a partir das características dos conjuntos de dados (meta-features). No entanto, os escores de desempenho que compõem o meta-alvo variam em escala e distribuição, levantando a questão de se técnicas de escalonamento aplicadas ao meta-alvo afetam o meta-modelo. Inspirados por Amorim et al. (2022), que demonstraram que a escolha do escalonamento importa para classificadores de nível base, este trabalho transpõe a questão para o nível meta. Avaliamos quatro meta-modelos (Ridge, SVR, Random Forest, MLP) combinados com cinco estratégias de escalonamento (Baseline, StandardScaler, MinMaxScaler, RobustScaler, QuantileTransformer) em quatro cenários experimentais: 50 conjuntos de dados controlados, 144 datasets reais do OpenML com 8 meta-features manuais, 200 datasets reais de famílias diferentes com 8 meta-features manuais, e os mesmos 200 datasets de famílias diferentes com 78 meta-features extraídas automaticamente pelo PyMFE. Os resultados mostram que a sensibilidade ao escalonamento é determinada pela arquitetura: o MLP colapsa sem escalonamento centralizado (Spearman cai a 0,03 com meta-features em dados reais), o SVR se beneficia para tarefas de ranqueamento, o Random Forest é invariante e o QuantileTransformer degrada consistentemente a capacidade de ranqueamento. Meta-features mais ricas reduzem substancialmente o erro de predição em todos os meta-modelos. Nenhum escalonador domina universalmente.*

1. Introdução

A seleção de algoritmos de aprendizado de máquina (ML) é uma das tarefas mais custosas na construção de sistemas inteligentes. Em vez de testar exaustivamente todos os candidatos para um novo dataset — inviável em tempo e recursos —, a *meta-aprendizagem* usa experiências passadas para treinar um meta-modelo que mapeia características dos datasets (*meta-features*) ao desempenho real dos algoritmos base (*meta-alvo*), permitindo recomendar a melhor opção para um novo cenário [Vanschoren 2019].

Embora as *meta-features* de entrada normalmente passem por pré-processamento cuidadoso, os escores de desempenho que compõem o *meta-alvo* costumam ser usados em sua forma bruta. Métricas como o F1-macro, porém, variam muito em escala e distribuição, e essa amplitude pode prejudicar meta-modelos sensíveis a essas variações. O trabalho de [Amorim et al. 2022] demonstrou, no nível base, que *"a escolha da técnica de escalonamento importa para a performance de classificação"* — este trabalho transpõe a mesma questão para o nível meta.

Problema: O escalonamento do meta-alvo em sistemas de meta-aprendizagem para seleção de algoritmos produz diferenças relevantes de performance? Existe uma recomendação geral que funcione para qualquer meta-modelo?

Hipóteses: (H1) Meta-modelos sensíveis ao escalonamento apresentarão diferença estatisticamente significativa entre as técnicas. (H2) Meta-modelos baseados em árvores serão insensíveis. (H3) Não existirá uma técnica universalmente superior.

Contribuições principais:

- Primeira investigação sistemática do impacto do escalonamento do *meta-alvo* (e não das meta-features) em sistemas de meta-aprendizagem;
- Demonstração de que o MLP, sem escalonamento centralizado, produz recomendações próximas ao acaso com dados reais, resultado revertido pelo StandardScaler ou RobustScaler;
- Identificação do QuantileTransformer como prejudicial à capacidade de ranqueamento na maioria dos meta-modelos;
- Validação em quatro cenários experimentais: (i) dados controlados; (ii) 144 datasets reais do OpenML; (iii) 200 datasets reais de famílias diferentes; e (iv) 200 datasets de famílias diferentes com 78 meta-features extraídas pelo PyMFE;
- Demonstração de que meta-features ricas (PyMFE) reduzem substancialmente o erro de predição.

2. Proposta

2.1. Formulação do Problema

Seja $\mathcal{D} = \{d_1, \ldots, d_N\}$ um conjunto de N datasets de classificação. Para cada d_i, extraímos um vetor de meta-features $\mathbf{x}_i \in \mathbb{R}^D$, formando a matriz $X \in \mathbb{R}^{N \times D}$. Avaliamos M algoritmos base $\mathcal{A} = \{a_1, \ldots, a_M\}$ em cada dataset, obtendo o vetor de performances $\mathbf{y}_i \in \mathbb{R}^M$ (F1-macro de cada a_j em d_i) e a matriz de meta-alvo $Y \in \mathbb{R}^{N \times M}$. O objetivo é treinar uma função f que preveja $\hat{\mathbf{y}}_i \approx \mathbf{y}_i$ a partir de $\mathbf{x}_i$, recomendando o algoritmo com melhor estimativa para um novo dataset.

2.2. Pipeline de Meta-Aprendizagem com Escalonamento do Alvo

A proposta central é estudar o efeito de uma transformação $S : \mathbb{R}^{N \times M} \to \mathbb{R}^{N \times M}$ aplicada sobre Y *antes* do treinamento. A Figura 1 ilustra o pipeline proposto.

2.3. Técnicas de Escalonamento Avaliadas

As cinco técnicas, aplicadas coluna a coluna sobre Y, são:

1. **Baseline** (sem escalonamento): $\tilde{y} = y$. Serve como grupo de controle, mantendo os valores de desempenho originais (F1-macro) sem qualquer modificação;
2. **StandardScaler** (Z-score): $\tilde{y} = (y - \mu)/\sigma$. Onde μ é a média e σ o desvio padrão. Centraliza os dados em torno de zero com variância unitária, auxiliando na convergência de métodos baseados em gradiente;
3. **MinMaxScaler**: $\tilde{y} = (y - y_{\min})/(y_{\max} - y_{\min})$. Mapeia linearmente os valores para o intervalo $[0, 1]$, preservando a forma da distribuição original e a proporção das distâncias entre os pontos;

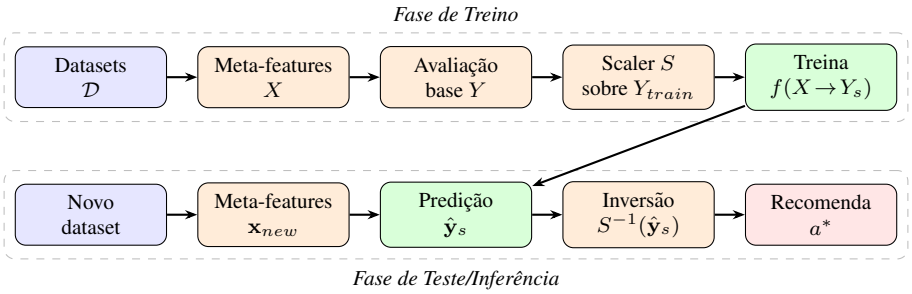


Figura 1. Pipeline de meta-aprendizagem com escalonamento do meta-alvo. O escalonador S é ajustado exclusivamente nos dados de treino, evitando *data leakage*. A inversão S^{-1} garante que as métricas finais sejam avaliadas na escala original do F1-score.

4. **RobustScaler**: $\tilde{y} = (y - \text{mediana})/\text{IQR}$. Onde IQR é o intervalo interquartil $(Q_3 - Q_1)$. Utiliza métricas robustas à presença de outliers (mediana e quartis), evitando que desempenhos anomalamente baixos ou altos distorçam a escala;
5. **QuantileTransformer**: $\tilde{y} = \Phi^{-1}\left(\frac{\text{rank}(y)}{n+1}\right)$. Onde Φ^{-1} é a função inversa da distribuição normal cumulativa e n é o número de amostras. Mapeia os dados de forma não-linear para uma distribuição normal, suavizando variações extremas, mas alterando as distâncias relativas entre os ranks.

As técnicas 1–4 são **monotônicas** (preservam a ordenação relativa), enquanto o QuantileTransformer é **não-linear** e pode distorcer a hierarquia de ranks entre algoritmos.

2.4. Métricas de Avaliação

O meta-modelo é avaliado em três dimensões:

- **MAE**: erro absoluto médio na escala original do F1-score;
- **Spearman** (ρ_s): correlação entre a ordenação real e a prevista dos M algoritmos; varia de -1 a $+1$;
- **Acc@1**: proporção de datasets em que o algoritmo mais recomendado é de fato o melhor.

3. Metodologia Experimental

3.1. Conjuntos de Dados

Foram conduzidos quatro experimentos:

Experimento 1 (controlado): 50 datasets de classificação — 4 reais da biblioteca `scikit-learn` (Iris, Wine, Breast Cancer, Digits) e 46 sintéticos gerados com `make_classification`, variando $N \in [200, 1000]$ instâncias, $D \in [6, 25]$ atributos, $C \in \{2, 3\}$ classes e Imbalance Ratio $\in [1{,}0, 20{,}0]$.

Experimento 2 (realista inicial): 144 datasets reais do OpenML [Vanschoren et al. 2014], selecionados por critérios de qualidade (sem valores ausentes, status ativo, dataset de referência).

Experimento 3 (realista expandido): 200 datasets reais do OpenML de famílias diferentes, garantindo que não haja redundância na distribuição de características das tarefas de classificação. Para isso, o catálogo foi filtrado eliminando duplicatas com base no prefixo do nome dos conjuntos de dados. O perfil das meta-features é apresentado na Tabela 1.

Experimento 4 (PyMFE): os mesmos 200 datasets de famílias diferentes do Experimento 3, mas substituindo as 8 meta-features manuais por 78 descritores extraídos automaticamente pelo `PyMFE` [Alcobaça et al. 2020] (grupos `general`, `statistical`, `info-theory` e `clustering`), com imputação por mediana e recorte de valores superiores a 10^{10}.

Tabela 1. Perfil das meta-features dos Experimentos 3 e 4 (200 datasets reais de famílias diferentes do OpenML).

Meta-feature	Mín.	Q1	Mediana	Q3	Máx.
# Instâncias (N)	21	187	351,0	712,0	2.000
# Atributos (D)	2	9	18,0	37,0	2.836
Razão D/N	0,002	0,023	0,048	0,096	7,144
Imbalance Ratio (IR)	1,00	1,17	2,10	5,31	534,0
Entropia	0,00	0,83	0,99	1,00	3,32
Pearson médio	0,00	0,08	0,17	0,34	0,78

3.2. Meta-features Extraídas

Nos Experimentos 1–3, foram calculadas 8 meta-features estáticas: número de instâncias, número de atributos, razão D/N, Imbalance Ratio, entropia de Shannon das classes, correlação média de Pearson (valor absoluto) entre atributos, variância média dos atributos e informação mútua média dos atributos com o alvo.

No Experimento 4, essas 8 features foram substituídas pelos 78 descritores do `PyMFE`, cobrindo quatro grupos: `general` (dimensionalidade, proporção de classes), `statistical` (correlação canônica, assimetria, variância), `info-theory` (entropia, informação mútua, concentração) e `clustering` (Calinski-Harabasz, Davies-Bouldin, Dunn).

Em todos os experimentos, as meta-features de entrada X foram padronizadas com StandardScaler ajustado exclusivamente no fold de treino.

3.3. Algoritmos Base e Meta-alvo

Cinco algoritmos base foram avaliados em cada dataset via validação cruzada estratificada de 5 folds, com F1-macro como métrica: **KNN** ($k = 5$), **Decision Tree** (CART), **Random Forest** (100 estimadores), **Gaussian Naive Bayes** e **SVM** (kernel RBF). Todos os hiperparâmetros mantiveram os valores padrão do `scikit-learn` 1.2.2, com *random state* fixado em 42.

3.4. Meta-modelos

Quatro meta-modelos foram avaliados como regressores multi-saída (prevendo o F1 dos 5 algoritmos simultaneamente):

- **Ridge** ($\alpha = 1{,}0$): modelo linear;
- **SVR** (kernel RBF, $C = 1{,}0$, $\varepsilon = 0{,}1$): modelo não-linear com kernel implícito;
- **Random Forest Regressor** (100 estimadores): ensemble baseado em árvores;
- **MLP Regressor** (camadas $[16, 8]$, solver Adam, `max_iter=2000`): rede neural.

3.5. Protocolo de Validação

Utilizou-se **validação cruzada de 5 folds no nível meta** (divisão por datasets). Em cada fold, o scaler do meta-alvo foi ajustado nos dados de treino, o meta-modelo foi treinado no alvo escalonado e as predições de teste foram invertidas para a escala original antes do cálculo das métricas — garantindo que não ocorra vazamento de dados (*data leakage*).

3.6. Testes Estatísticos

Para verificar a significância das diferenças entre scalers, empregamos:

- **Teste de Friedman** (global): verifica se ao menos um scaler difere dos demais por meta-modelo, usando os vetores de MAE por dataset ($\alpha = 0{,}05$);
- **Wilcoxon signed-rank** (post-hoc): compara cada scaler contra o Baseline par a par.

4. Resultados e Discussão

4.1. Distribuição do Meta-Alvo

A Tabela 2 mostra os dados descritivos do meta-alvo nos 200 datasets reais. O desvio padrão alto indica que a variabilidade é grande: existem conjuntos de dados fáceis, onde o F1-score fica perto de 1,0, e outros bem difíceis, com F1-score perto de 0,0. Essa variação é o que torna o escalonamento tão importante.

Tabela 2. Distribuição do F1-macro (meta-alvo) para os 5 algoritmos base nos 200 datasets reais.

Algoritmo	Média	Desv. Pad.	Mediana	Máx.
KNN	0,654	0,197	0,643	1,000
Decision Tree	0,720	0,185	0,702	1,000
Random Forest	0,734	0,191	0,735	1,000
Gaussian NB	0,643	0,210	0,648	1,000
SVM	0,589	0,239	0,555	1,000

4.2. Resultados por Experimento

Para entender melhor as descobertas sem se perder em detalhes, reunimos as discussões dos quatro experimentos a seguir. Assim, conseguimos ver como o comportamento dos modelos mudou ao passarmos de dados sintéticos controlados para dados reais e complexos.

O Experimento 1 (com 50 datasets controlados) serviu como ponto de partida e confirmou algumas suposições: os modelos Ridge e Random Forest não sofreram impacto

com escalonadores lineares, o SVR melhorou um pouco em termos de erro e ordenação, e o MLP falhou completamente quando os dados não foram centralizados.

Os Experimentos 2 e 3 (com 144 e 200 datasets reais e 8 meta-features) confirmaram esses comportamentos em cenários de dados reais e variados. O colapso do MLP foi ainda mais nítido aqui: no Experimento 2, a correlação de Spearman despencou para 0,034 (indicando nenhuma capacidade de ordenação) e a acurácia de recomendação Acc@1 caiu para 12,5% (o que é pior que o acaso de 20% para 5 opções). Em contrapartida, usar StandardScaler ou RobustScaler recuperou a capacidade de previsão do modelo, com o MAE caindo para a faixa de 0,11 a 0,13 e a correlação de Spearman subindo para valores entre 0,41 e 0,45. A Tabela 5 na Seção 4.4.4 resume o desempenho do MLP ao longo dos quatro testes.

O Experimento 4 (com 200 datasets e 78 meta-features do PyMFE) traz os resultados do pipeline mais completo do estudo. O uso dessas características mais ricas reduziu bastante o erro de previsão de todos os modelos. Por exemplo, o menor MAE obtido pelo SVR caiu de 0,123 (no Experimento 3) para 0,088 (no Experimento 4), e o do Random Forest baixou de 0,114 para 0,085. A Tabela 3 resume os resultados desse cenário final, comparando as baselines com as melhores configurações de escalonamento.

Tabela 3. Resumo dos resultados do Experimento 4 (PyMFE, 200 datasets). Comparativo entre Baseline e escalonadores recomendados.

Meta-Modelo	Técnica	MAE	Spearman	Acc@1
Ridge	Baseline / Scalers Lineares	**0,1007**	**0,4966**	**0,3788**
	QuantileTransformer	0,1152	0,4483	0,3586
SVR	Baseline	0,0978	0,5213	0,3687
	StandardScaler (Melhor Rank)	0,0898	**0,5982**	**0,4949**
	RobustScaler (Melhor MAE)	**0,0884**	0,5966	0,4646
Random Forest	Baseline	0,0860	0,6019	0,4747
	MinMaxScaler (Melhor Geral)	**0,0845**	**0,6157**	**0,5152**
MLP	Baseline	0,1830	0,0802	0,2121
	StandardScaler (Melhor MAE)	**0,1165**	0,3875	0,3384
	RobustScaler (Melhor Acc@1)	0,1220	0,3542	**0,3586**
	QuantileTransformer (Melhor ρ_s)	0,1294	**0,4355**	0,2980

4.3. Testes de Hipóteses

Teste de Friedman. Este teste analisa se há alguma diferença estatisticamente significativa no erro absoluto (MAE) dos modelos ao variarmos os escalonadores (os resultados estão na Tabela 4). Nos Experimentos 2 e 3, apenas o MLP mostrou diferenças significativas, o que indica que os modelos Ridge, SVR e Random Forest conseguem lidar bem com os dados brutos. No entanto, no Experimento 4 (com PyMFE), a maior quantidade de características deu mais precisão para a análise: o Ridge e o SVR também passaram a apresentar diferenças estatísticas significativas. Por outro lado, o Random Forest continuou muito estável e não sofreu impacto estatístico sob nenhuma técnica.

Tabela 4. Resultados do teste de Friedman por meta-modelo nos Experimentos 2, 3 e 4.

Experimento	Meta-Modelo	χ^2	p-value	Decisão
Exp. 2 (144 ds)	Ridge	3,04	0,551	Não significativo
	SVR	8,37	0,079	Não significativo
	Random Forest	5,41	0,248	Não significativo
	MLP	**148,72**	$\mathbf{3{,}8 \times 10^{-31}}$	**Significativo**
Exp. 3 (200 ds)	Ridge	1,61	0,807	Não significativo
	SVR	3,41	0,491	Não significativo
	Random Forest	0,34	0,987	Não significativo
	MLP	**110,71**	$\mathbf{5{,}1 \times 10^{-23}}$	**Significativo**
Exp. 4 (PyMFE)	Ridge	**22,38**	$\mathbf{1{,}7 \times 10^{-4}}$	**Significativo**
	SVR	**53,92**	$\mathbf{5{,}5 \times 10^{-11}}$	**Significativo**
	Random Forest	4,58	0,333	Não significativo
	MLP	**278,02**	$\mathbf{5{,}9 \times 10^{-59}}$	**Significativo**

Wilcoxon post-hoc para o MLP. Os testes de comparação par a par (Wilcoxon) confirmaram que qualquer escalonamento é muito melhor do que deixar o meta-alvo do MLP sem tratamento nos três experimentos com dados reais ($p < 10^{-4}$). O StandardScaler e o RobustScaler diminuíram o erro absoluto em até 36% e ajudaram o modelo a ordenar melhor as opções (a correlação de Spearman subiu de valores quase nulos para mais de 0,40). Por exemplo, no Experimento 4, o StandardScaler reduziu o MAE do MLP de 0,183 para 0,117 e elevou a correlação de Spearman de 0,080 para 0,388.

4.4. Análise por Meta-Modelo

4.4.1. Ridge Regression: Invariância a Transformações Lineares

O modelo Ridge apresentou resultados *idênticos* para os escalonadores lineares (StandardScaler, MinMaxScaler e RobustScaler) em todos os quatro experimentos. Isso acontece por conta de uma característica matemática simples: quando aplicamos uma transformação linear como $\tilde{y} = a \cdot y + b$, o Ridge apenas ajusta os coeficientes para compensar a escala. Ao fazer a inversão do escalonamento na saída, o resultado final volta a ser idêntico ao original. A única exceção é o QuantileTransformer. Como ele aplica uma transformação não-linear, a ordem dos dados no espaço-alvo é distorcida e a inversão não consegue recuperar a escala original, o que explica a queda na acurácia de recomendação (Acc@1) observada em todos os cenários.

4.4.2. SVR: O Escalonamento Ajuda Mais na Recomendação do que na Redução do Erro

Para o SVR, o escalonamento faz uma grande diferença na qualidade das recomendações (Acc@1 e correlação de Spearman) em todos os cenários, embora o impacto no erro absoluto (MAE) varie de acordo com o experimento. Em dados reais sem o PyMFE (Experimentos 2 e 3), o erro mudou muito pouco, mas a capacidade de recomendar o melhor algoritmo aumentou significativamente: no Experimento 2, o Acc@1 subiu

de 0,486 para 0,563 com RobustScaler, e no Experimento 3 foi de 0,429 para 0,470. Já no Experimento 4, com as meta-features mais completas do PyMFE, o modelo melhorou em todas as frentes: o MAE caiu cerca de 10% (de 0,098 para 0,088 com RobustScaler) e a acurácia de recomendação foi de 0,369 para 0,495 com StandardScaler.

4.4.3. Random Forest: Robustez ao Escalonamento

O Random Forest confirmou ser o modelo mais resistente a mudanças de escala no meta-alvo. Em todos os experimentos, a variação de MAE entre as técnicas lineares foi minúscula (menor que 0,007), e os testes estatísticos não indicaram diferença significativa global. Essa estabilidade acontece pela própria natureza dos modelos de árvore, que tomam decisões de divisão baseadas na ordenação relativa (ranks) dos dados, mantida por qualquer transformação linear. A única técnica que atrapalha o Random Forest é o QuantileTransformer: por alterar a ordem relativa dos dados, ele prejudicou a capacidade de recomendação (Acc@1) em todos os cenários testados.

4.4.4. MLP: Colapso sem Escalonamento Centralizado

O resultado mais marcante é o comportamento do MLP. Sem usar um escalonador centralizado no meta-alvo, a rede neural simplesmente falha em aprender qualquer padrão útil nos dados reais. A correlação de Spearman cai para patamares quase nulos (0,034 no Exp. 2, 0,167 no Exp. 3 e 0,080 no Exp. 4) e a acurácia de recomendação fica em torno ou abaixo do acaso de 20% (chegando a apenas 12,5% no Exp. 2). A Tabela 5 detalha esse colapso nos quatro cenários:

Tabela 5. Comportamento do MLP nos quatro experimentos: Baseline versus melhor scaler.

Experimento	Scaler	MAE	Spearman	Acc@1
Exp. 1 (sintético, 50 ds)	Baseline	0,147	0,274	0,320
	RobustScaler	**0,071**	**0,588**	**0,440**
Exp. 2 (real, 144 ds)	Baseline	0,179	0,034	0,125
	RobustScaler	**0,118**	**0,453**	**0,438**
Exp. 3 (real, 200 ds)	Baseline	0,174	0,167	0,207
	StandardScaler	**0,131**	**0,431**	0,374
	RobustScaler	0,135	0,413	**0,399**
Exp. 4 (PyMFE, 200 ds)	Baseline	0,183	0,080	0,212
	QuantileTransformer	**0,129**	**0,436**	0,298
	RobustScaler	0,122	0,354	**0,359**

Esse colapso é muito forte com dados reais porque o meta-alvo apresenta alta variância e alguns valores extremos de desempenho (algoritmos com F1-score próximo de 0 ou de 1 no mesmo conjunto de dados). Quando esses dados não são centralizados em torno de zero, os pesos da rede neural sofrem com gradientes instáveis ou saturação

das funções de ativação. Técnicas como StandardScaler e RobustScaler salvam o modelo porque centralizam e limitam a dispersão dos dados.

O MinMaxScaler não ajuda o MLP porque ele apenas mapeia os dados para o intervalo $[0, 1]$, mantendo a assimetria e a falta de centralização (o que explica por que ele dá resultados quase idênticos ao Baseline sem escalonamento). Por fim, no Experimento 4 (com as 78 características do PyMFE), um comportamento curioso: o QuantileTransformer deu a melhor correlação de Spearman do MLP (0,436). Isso mostra que, em espaços com características mais densas e estruturadas, essa transformação não-linear por quantis pode ajudar a rede neural a identificar melhor a hierarquia entre os algoritmos base.

4.5. O Paradoxo do QuantileTransformer

Embora o QuantileTransformer possa ajudar a reduzir o erro absoluto (MAE) ao transformar os dados em uma distribuição normal, ele costuma piorar a capacidade de recomendação (Acc@1) dos modelos. Isso acontece porque a sua transformação não-linear muda as distâncias relativas entre os scores de F1 dos algoritmos. Por exemplo, se dois algoritmos têm desempenhos muito parecidos (como 0,70 e 0,71), o QuantileTransformer pode afastar bastante os seus valores transformados, enquanto dois desempenhos distantes (como 0,50 e 0,70) podem ficar mais próximos. Essa distorção dificulta que a inversão do escalonamento recupere a ordem exata das previsões.

Se a intenção for de fato **recomendar o melhor algoritmo**, o QuantileTransformer deve ser evitado na maioria dos modelos devido à sua tendência de degradar o ranqueamento. Por outro lado, essa técnica destaca-se como a mais eficaz para lidar com dados desbalanceados, uma vez que sua capacidade de mapear comportamentos não lineares suaviza o impacto de distribuições muito assimétricas. A única exceção a essa perda generalizada de ordenação ocorreu com o MLP no Experimento 4 (com PyMFE), no qual a maior riqueza das meta-features parece ter ajudado a compensar as distorções introduzidas pelo escalonador.

4.6. Consistência entre os Experimentos

A Tabela 6 sintetiza a consistência dos achados ao longo dos quatro experimentos, mostrando que os padrões identificados com dados controlados são robustos a cenários reais heterogêneos.

Tabela 6. Consistência dos achados ao longo dos quatro experimentos.

Fenômeno observado	Exp. 1	Exp. 2	Exp. 3	Exp. 4	Avaliação
Ridge invariante a scalers lineares	✓	✓	✓	✓	Alta
Ridge degradado pelo QT (Acc@1)	✓	✓	✓	✓	Alta
MLP: colapso sem escalonamento centralizado	Parcial	Total	Total	Total	Reforçada
MLP: ganho no MAE com SS/RS	51,8%	34,3%	24,9%	36,3%	Alta
MLP: MinMaxScaler sem ganho	✓	✓	✓	✓	Alta
RF invariante a scalers lineares	✓	✓	✓	✓	Alta
QT degrada rankings em todos os modelos	✓	✓	✓	✓	Alta
SVR beneficiado pelo escalonamento	MAE	Spearman	Spearman	Todos	Alta

5. Conclusão

Este estudo analisou como o escalonamento do meta-alvo influencia o desempenho de sistemas de meta-aprendizagem na tarefa de selecionar algoritmos. Para isso, realizaram-se quatro experimentos: 50 datasets em ambiente controlado, 144 datasets reais (etapa inicial), 200 datasets reais de famílias diferentes (etapa expandida) e, finalmente, 200 datasets reais de famílias diferentes utilizando 78 meta-features extraídas automaticamente pelo PyMFE.

Os resultados deixam claro que **a sensibilidade ao escalonamento do meta-alvo depende muito mais da arquitetura do meta-modelo** do que do tipo ou da origem dos dados analisados:

- **Ridge**: é imune a transformações lineares no alvo por conta de sua formulação matemática. Apenas escalonamentos não-lineares conseguem alterar o seu comportamento.
- **SVR**: melhora bastante a qualidade das recomendações (acurácia Acc@1 e Spearman) ao usar StandardScaler ou RobustScaler. Essa vantagem fica ainda mais nítida e estatisticamente significativa à medida que se usa características de entrada mais ricas (como as do PyMFE).
- **Random Forest**: mostrou-se muito estável e quase não sofreu impactos com os escalonadores lineares, sendo o modelo mais seguro para uso geral. Curiosamente, ao combinar o Random Forest com MinMaxScaler no Experimento 4, obteve-se a maior correlação de Spearman de todo o estudo (0,616).
- **MLP**: o escalonamento centralizado (usando StandardScaler ou RobustScaler) é obrigatório. Sem esse ajuste, a rede neural falha nos cenários reais, dando palpites equivalentes a um sorteio aleatório (a correlação de Spearman fica abaixo de 0,09 no Experimento 4).

O **QuantileTransformer deve ser evitado se o objetivo principal for otimizar a qualidade das recomendações** (Acc@1), uma vez que tende a degradar a ordenação. No entanto, ele se destaca como a escolha mais eficaz na presença de dados altamente desbalanceados, além de ter melhorado a correlação de Spearman do MLP no Experimento 4 com PyMFE.

A adoção de meta-features mais completas (PyMFE) reduziu drasticamente o erro de previsão em todos os modelos.

Por fim, confirma-se a hipótese H3 de que não existe um escalonador perfeito para todas as situações. A melhor escolha sempre vai depender do modelo que você está usando e do seu objetivo principal (seja reduzir o erro numérico ou obter as melhores recomendações). Como regra geral e prática, recomenda-se o uso do **RobustScaler**, pois ele oferece o melhor equilíbrio entre precisão e estabilidade, especialmente ao lidar com dados reais e que possuem desempenhos muito variados.

Referências

Alcobaça, E., Siqueira, F., Rivolli, A., Garcia, L. P. F., Oliva, J. T., and de Carvalho, A. C. P. L. F. (2020). MFE: Towards reproducible meta-feature extraction. *Journal of Machine Learning Research*, 21(111):1–5.

Amorim, G., Santos, M., de Oliveira, B., et al. (2022). The choice of scaling technique matters for classification performance. *Applied Soft Computing*, 119:108643.

Vanschoren, J. (2019). Meta-learning: A survey. In Hutter, F., Kotthoff, L., and Vanschoren, J., editors, *Automated Machine Learning: Methods, Systems, Challenges*, pages 35–61. Springer.

Vanschoren, J., van Rijn, J. N., Bischl, B., and Torgo, L. (2014). OpenML: Networked science in machine learning. *ACM SIGKDD Explorations Newsletter*, 15(2):49–60.